

UNIT 1: Introduction to programming languages

Programming languages

Programming languages are tools that enable humans (**programmers/developers**) to communicate with computers, providing instructions that machines can interpret and execute. A programming language is a formal set of instructions and syntax that enables programmers to communicate with computers, allowing them to create software applications and control computer behavior.

These languages provide the necessary framework for writing code, which is then executed by computers to perform specific tasks. **Over time, programming languages have evolved significantly to accommodate the growing complexity of tasks and advancements in computing technology.**

Types of programming languages **based on abstraction levels:**

A. Low level language

Low level language are machine dependent language that means, program written in low level language for one processor may not execute on another processor. Moreover, programmer need to have detail knowledge about computer hardware architecture.

It makes programming in low level language much more time consuming and difficult. There are two types of low level language. ***They are Machine level language and Assembly language.***

1. Machine level language

- Machine Level language is also known as first generation language (1GL).
- Since, MLL use machine code i.e Binary Digits (0 and 1) so, it is directly understood by the computer and doesn't require any language translator.
- The program execution time in MLL is faster. MLL is also machine dependent which means programs written in machine level language for one type of processor doesn't run on another processor.

Advantages of machine level language

- MLL is the only language which is directly understood by the computer.
- Since, it is directly understood, language processor is not required.
- Program execution is faster as no conversion is required.

Disadvantages of machine level language

- Program development is difficult and time-consuming.
- It is machine dependent, so program developed in MLL may not run on several other processor.
- Detail hardware architecture knowledge is required.

2. Assembly Language

- Assembly language also known as second generation language (2GL) is also the type of low level language which is very much closer to machine level language.
- In Assembly language we use mnemonics code i.e. instruction in the form of abbreviation such as ADD, SUB, DIV, JMP, MOV etc. instead of directly using Binary digits.

- Since, Assembly language is not directly understood by the computer we use language processor such as assembler to convert program written in assembly language in machine level language.
- Here, program written in assembly language is source program whereas after conversion it becomes object program.

Advantages of assembly language

- Easier to remember codes compared to MLL.
- Program development and debugging is faster and easier.
- It can be used to develop hardware oriented software.

Disadvantages of assembly language.

- Language translator is required as it is not directly understood by the computer.
- Since it is low level language, it is also machine dependent.
- Program execution time is slower compared to MLL.

B. High level language

High level language are machine independent language that means program written in high level language for one processor can run on several other processor.

Programmer don't need to have detail knowledge about computer hardware architecture. It makes programming in high level language much easier and efficient.

High level language use instruction which is very closer and similar to English words and mathematical notations. ***There are many types of high level language. Some of them are Procedural oriented language and Object oriented language etc.***

1. Procedural Oriented Language/Functional languages /Function Oriented Languages:

- Procedural language is commonly used high level language which primarily focus on process to solve a particular problem.
- It used simple English like statements and mathematical notation which makes it easier to learn and use.
- The statement written in procedural oriented language follows top to bottom approach, which means every statement written are executed sequentially from top left to right bottom.
- Since, procedural language are primitive high level language they do not support advance modern features such as inheritance, polymorphism, encapsulation and so on.
- Program are divided into number of functions to solve several problem. C, FORTAN, BASIC, Pascal are some of the popular procedural oriented language.

Advantages of procedural oriented language

- It is general purpose programming language.
- It is easier and faster to develop programs compared to low level language.
- Procedural language are advance and user friendly.
- It is machine independent which makes program deployment easier among large number of devices.

Disadvantages of procedural oriented language

- It need language translator such as compiler and interpreter to make it machine understandable.
- Program conversion makes its execution slower.
- It is difficult to develop hardware oriented program.

2. Object Oriented Programming Language

- Object oriented language generally known as problem oriented language. It is often referred to as non-procedural language.
- Unlike procedural it doesn't focus on process rather its primarily focuses on data and object.
- OOP main concept lies on class and object where, object consist of data and member function and class is comprised of similar objects.
- OOP are not executed sequentially rather they follow bottom up approach.
- Object oriented language are the modern high level language so, it support several advance features such as data hiding, inheritance, polymorphism etc.
- C++, C#, PHP, Python etc. are the widely used OOP.

Advantages of object oriented language

- Program development is faster as code can be reused.
- Used in web based application and game development
- Maintains high level of security features.

Disadvantages of object oriented language

- Language processor is required to execute the program.
- Advance programming skill is required to write program in OOP.
- Program execution can be slower as it needs conversion.

Generations of Programming Languages

The evolution of programming languages is categorized into five distinct generations, each representing significant advancements in technology and programming paradigms. These generations reflect the increasing abstraction from machine code and the growing focus on usability for programmers.

First Generation (1940s - 1950s)

- The first generation consists of machine language, which is the most basic form of programming.
- It uses binary code (0s and 1s) that is directly executed by a computer's CPU.
- Extremely difficult for humans to read and write due to its binary nature.
- Examples: No specific named examples exist as it varies by hardware.
- Impact: Established the foundational principles of computer programming in a cumbersome and error-prone manner.

Second Generation (1950s - 1960s)

- Known as assembly language, this generation introduced mnemonic codes or symbols to represent machine instructions, making it slightly more human-readable.
- Examples: NASM (Netwide Assembler), MASM (Microsoft Macro Assembler).
- Impact: Made programming more accessible while still requiring a deep understanding of hardware specifics.

Third Generation (1960s - 1970s)

- This generation **brought forth** (Introduces) high-level programming languages, which abstracted away much of the hardware-specific details and used syntax closer to human languages.
- Readability was improved significantly, allowing for complex operations with simpler statements.
- Examples: C, FORTRAN, COBOL, Pascal, BASIC.
- Impact: Greatly increased productivity and accessibility in software development, enabling the creation of complex systems.

Fourth Generation (1980s - 1990s)

- Fourth-generation languages (4GLs) are designed for higher productivity and simplicity, often focusing on specific problem domains such as database management or report generation.
- Readability was closer to natural language, emphasizing ease of use.
- Examples: SQL, MATLAB, SAS, Visual Basic.
- Impact: Enhanced productivity in specialized areas like data analysis and database management

Fifth Generation (1980s - Present)

- Fifth-generation languages focus on problem-solving using constraints rather than traditional algorithms.
- They are often associated with artificial intelligence and logic programming.
- Readability Utilizes logic and declarative paradigms, allowing users to define goals rather than specific procedures.
- Examples: Prolog, Lisp, Mercury.
- Impact: Advanced the field of artificial intelligence and complex problem-solving systems

Structured programming:

Structured programming is a programming paradigm aimed at improving the clarity, quality, and development time of software. It emphasizes dividing programs into manageable and reusable pieces using **control structures**, **modularity**, and **logical organization**. **Key principles include sequence, selection, and iteration, avoiding the use of unstructured constructs like goto statements.**

Key Concepts of Structured Programming

A. Control Structures: Control structures are some pre-defined structures which control the flow of execution of the C program.

Structured programming relies on three primary control structures:

- **Sequence:** This involves executing statements in a linear order, one after the other.
- **Selection:** This allows for decision-making within the code, typically using constructs like if, else, and switch statements. The program executes different paths based on conditions.
- **Iteration:** This involves repeating a block of code until a certain condition is met, using loops such as for, while, or do-while.

B. Modularization

A significant aspect of structured programming is the use of subroutines (or functions), which are self-contained blocks of code designed to perform specific tasks. This modular approach enhances code reusability and maintainability by allowing developers to break down complex problems into smaller, manageable units.

C. Block Structures

C supports block structures through the use of curly braces {} to group statements. This allows developers to define scopes for variables and control flow, ensuring that related statements are treated as a single unit.

Advantages of Structured Programming

Improved Readability: Code organized into clear structures is easier to read and understand.

Easier Maintenance: With well-defined modules, updating or debugging code becomes simpler.

Reduced Complexity: By avoiding unstructured jumps (like goto statements), structured programming reduces the potential for errors and makes logic flow more predictable.

Enhanced Collaboration: Structured programs facilitate teamwork as different programmers can work on separate modules without causing conflicts in the overall program structure.

Computer software

Computer software refers to a collection of instructions, data, or programs that enable computers to perform specific tasks. Unlike hardware, which encompasses the physical components of a computer, software is intangible and exists as code written in programming languages.

Types of Software:

1. System Software:

System software is a collection of programs designed to manage and control computer hardware so that application software can run. It acts as a bridge between the hardware and application programs, enabling them to work together efficiently. System software is crucial for the computer's basic functions and operation.

Key Characteristics of System Software:

1. **Essential for Computer Operation:** Without system software, the hardware cannot function or run application programs.
2. **Interacts Directly with Hardware:** System software interacts closely with hardware to manage resources such as memory, CPU, storage, and input/output devices.
3. **Background Operations:** Most system software runs in the background, invisible to the user, ensuring smooth and efficient operation of the computer system.
4. **Automatic and Pre-configured:** System software typically runs automatically when the computer is powered on and does not require constant user intervention

Types of System Software:

1. Operating System (OS):

- The operating system is the most critical piece of system software, responsible for managing all hardware resources and providing a user interface to interact with the computer.
- It serves as a platform for running application software and performs vital tasks like memory management, process scheduling, file handling, and hardware control.
- **Examples of Operating Systems includes** Windows, macOS, Linux (Ubuntu, Red Hat), Android, iOS etc.

2. Device Drivers:

- Device drivers are small software programs that allow the operating system to communicate with hardware devices.
- Without drivers, the OS cannot control or interact with the hardware effectively.
- Each hardware component, such as a printer, keyboard, graphics card, or network adapter, requires its own specific driver.
- **Functions of Device Drivers** Enable plug-and-play functionality for easy connection of new hardware.
- **Examples of Device Drivers includes** Printer drivers (e.g., for HP or Canon printers), Graphics drivers (e.g., NVIDIA or AMD drivers), Network drivers (e.g., Wi-Fi or Ethernet drivers).

3. Utility Software:

- Utility software includes a variety of small programs that help manage, maintain, and optimize computer performance.
- These programs focus on specific tasks, such as system security, performance optimization, or file management.
- Utilities often provide users with the ability to troubleshoot and improve the system's health.

Examples of Utility Software and Their Functions:

- a. **Antivirus Software:** Protects against malicious software (e.g., Norton, McAfee).
- b. **Disk Cleanup Tools:** Frees up disk space by removing unnecessary files (e.g., CCleaner).

- c. **Backup Software:** Ensures data is backed up regularly (e.g., Acronis True Image).
- d. **System Monitors:** Tracks and reports on system performance, such as CPU usage, memory, and temperature (e.g., Task Manager in Windows, Activity Monitor in macOS).
- e. **File Compression Tools:** Reduce file sizes for easier storage or transfer (e.g., WinRAR, 7-Zip).
- f. **Disk Defragmenters:** Reorganize fragmented files on hard drives to optimize performance (e.g., Windows Disk Defragmenter).

2. Application Software:

Application software refers to programs designed to perform specific tasks or solve particular problems for the user. Unlike system software, which helps manage the hardware and basic operations of a computer, application software is created to help users accomplish tasks like document creation, media editing, web browsing, gaming, and more.

Key Characteristics of Application Software:

- **User-Centric:** Application software is designed to solve specific user problems or perform certain tasks.
- **Specific Purpose:** Each application is built with a particular objective in mind, such as document creation, media editing, communication, or gaming.
- **Variety of Formats:** Application software can range from simple, standalone programs to complex systems used in enterprise environments.
- **Requires System Software:** Application software relies on system software to function, as it requires an operating system and hardware resources to run.

Types of Application Software:

1. **Productivity Software** Productivity software is designed to help users perform tasks related to creating, managing, and organizing documents, presentations, spreadsheets, and databases. These tools are commonly used in personal, academic, and professional environments.

Key Types of Productivity Software:

- A. **Word Processors:** Used for creating, editing, formatting, and printing text documents.
 - a. **Examples:** Microsoft Word, Google Docs, LibreOffice Writer.
- B. **Spreadsheets:** Used for organizing, analyzing, and calculating data in tabular form.
 - a. **Examples:** Microsoft Excel, Google Sheets, LibreOffice Calc.
- C. **Presentation Software:** Used for creating visual presentations, often for meetings, lectures, or pitches.
 - a. **Examples:** Microsoft PowerPoint, Google Slides, Keynote.
- D. **Database Management Systems (DBMS):** Used for organizing, storing, and managing large amounts of data.
 - a. **Examples:** Microsoft Access, MySQL, Oracle Database.

2. **Media Software** Media software is designed to create, edit, and play multimedia content, such as images, videos, and audio files. It is widely used in creative industries, entertainment, and personal media management.

Key Types of Media Software:

- A. **Image Editing Software:** Used for manipulating digital images, creating graphics, and performing tasks like retouching and drawing.
 - a. **Examples:** Adobe Photoshop, GIMP, CorelDRAW.
- B. **Video Editing Software:** Used for editing and creating video content by combining video clips, effects, and audio.
 - a. **Examples:** Adobe Premiere Pro, Final Cut Pro, DaVinci Resolve.
- C. **Audio Editing Software:** Used for editing and producing sound recordings.
 - a. **Examples:** Audacity, Adobe Audition, FL Studio.
- D. **Media Players:** Used to play multimedia content like music, video, and images.
 - a. **Examples:** VLC Media Player, Windows Media Player, iTunes.

3. **Web Browsers** Web browsers are applications that allow users to access and navigate the World Wide Web. They interpret and display web pages, allowing users to interact with the internet by entering URLs, searching for content, and interacting with web-based applications.

Examples of Web Browsers:

- Google Chrome, Mozilla Firefox, Safari, Microsoft Edge etc.

4. **Business Software** Business software is used to automate, streamline, and optimize business operations. These applications are crucial for tasks such as accounting, managing customer relations, and tracking business resources.

Key Types of Business Software:

- A. **Enterprise Resource Planning (ERP):** Software used to manage and integrate key business processes across various departments.
 - a. **Examples:** SAP, Oracle ERP, Microsoft Dynamics.
- B. **Customer Relationship Management (CRM):** Software used to manage customer interactions, track sales, and improve marketing efforts.
 - a. **Examples:** Salesforce, HubSpot, Zoho CRM.
- C. **Accounting Software:** Used for managing financial transactions, bookkeeping, and financial reporting.
 - a. **Examples:** QuickBooks, Xero, FreshBooks.
- D. **Project Management Software:** Helps businesses plan, track, and manage projects from start to finish.
 - a. **Examples:** Trello, Asana, Microsoft Project.

Problem-solving using computers

Problem-solving using computers is the process of designing, developing, and implementing solutions to real-world or computational problems by leveraging computer systems and programming.

This systematic approach uses computational thinking, which involves breaking down a problem into smaller, manageable components and solving those using algorithms, logic, and coding. The steps required to solve problem using computer are described below.

1. Problem Analysis: The first step focuses on thoroughly understanding the problem and its requirements. This stage lays the foundation for the entire project. **The key activities are:**

- **Understanding the Problem:**
 - Define the problem clearly, asking, “What are we trying to solve?”
 - Identify key inputs and outputs. For example, if you’re designing a weather prediction app, inputs might be historical weather data, and outputs could be daily temperature forecasts.
- **Feasibility Analysis:**
 - Assess the practicality of the project in terms of technical capabilities, time, and cost. For instance, ask: "Do we have the data, tools, and expertise to implement the solution?"
 - Ensure the problem is worth solving and aligns with organizational goals.
- **Requirement Analysis:**
 - List all the required features and constraints in detail (e.g., speed, memory usage, or user-friendliness).
 - Requirements are divided into two types:
 - **Functional Requirements:** Specific tasks the program must perform, like sorting data.
 - **Non-functional Requirements:** Performance metrics, such as response time, scalability, or ease of use.

2. Design: The design phase focuses on planning how the program will work. This is the blueprint for coding. Design ensures clarity and eliminates logical flaws early in the process.

The key Activities are:

- **Algorithm Design:**
 - An algorithm is a step-by-step procedure to solve the problem. It helps ensure logical flow and efficiency.
 - Write a detailed, step-by-step procedure for solving the problem.
 - *Use pseudocode for clarity, which is an informal way to describe logic without focusing on syntax.*
- **Flowchart:**
 - A graphical representation of the algorithm. Flowcharts use symbols to show processes, decisions, and data flow, making it easier to visualize and understand the solution.
 - Create a visual representation of the steps in your algorithm.

3. Coding: The coding stage translates the design into a programming language code that a computer can execute. *The key activities are:*

- **Writing Code:**
 - Follow the algorithm and flowchart to ensure accuracy.
 - Choose the appropriate programming language (e.g., Python for data science, Java for cross-platform applications).
- **Compilation/Interpretation:** Depending on the programming language, the code is either compiled into machine language or interpreted line-by-line for execution.
 - **Compiled languages (e.g., C, C++):** The entire code is converted to machine language before execution.
 - **Interpreted languages (e.g., Python, JavaScript):** Code is converted to machine language line-by-line during execution.
- **Execution:**
 - Run the program to see if it works as intended. This is the first chance to catch runtime errors or basic bugs.

4. Testing and Debugging: Testing ensures the program functions correctly under various conditions, while debugging involves identifying and fixing errors. A robust testing and debugging process is critical for delivering a reliable program. *The key activities are:*

- **Testing:** Perform various tests to ensure the program behaves as expected. Testing can be classify into various types, as following:
 - **Unit Testing:** Test small, individual parts of the program (e.g., a single function).
 - **Integration Testing:** Ensure different components work well together.
 - **System Testing:** Test the entire system in a real-world environment.
 - **User Acceptance Testing (UAT):** Let end-users test the system to verify it meets their needs.
- **Debugging:** Identify and fix errors, the types of errors we encounter during Debugging are as following:
 - **Syntax Errors:** Mistakes in code syntax (e.g., missing semicolons in C++).
 - **Logical Errors:** The program runs but produces incorrect results due to flaws in logic.
 - **Runtime Errors:** Errors that occur while the program is running (e.g., dividing by zero).

5. Implementation: After the program is tested, it is deployed into the production environment for real-world use. *The key activities involves:*

- **Installation:**
 - Deploy the software on user systems or servers.
 - Configure hardware or network settings if required.
- **Training:**
 - Provide users with the necessary training or guides.
 - For example, for a hospital management system, train staff on how to input patient details.

6. Evaluation and Maintenance: Once implemented, the program needs regular monitoring to ensure it performs well and evolves with changing requirements. Maintenance ensures the longevity and relevance of the program. *The key activities are:*

- **Evaluation:**

- Collect feedback from users and assess the system's effectiveness (e.g., Are users satisfied? Are goals achieved?).
- Measure performance metrics, like execution time, error rate, or user satisfaction scores.

- **Maintenance:**

- Fix bugs that arise post-deployment.
- Make updates to accommodate new features, technologies, or requirements.
 - Example: Adding a mobile-friendly version of an application.
- Optimize performance over time (e.g., improving processing speed).

7. Program Documentation: Documentation is essential for long-term usability and maintainability. Good documentation prevents misunderstandings and ensures seamless collaboration and maintenance. Documentation is divided in following types:

- **User Documentation:**

- Guides for end-users to operate the system effectively.
- Example: A user manual for a banking app that explains how to check balances or transfer money.

- **Technical Documentation:**

- In-depth details for developers, such as:
 - System architecture.
 - Algorithms used.
 - Code explanations and comments.
- This documentation helps other programmers update or modify the system in the future.

By following these steps meticulously, you ensure the program is developed in a structured, efficient, and effective manner, meeting both immediate and long-term needs.

Algorithm:

An **algorithm** is a step-by-step procedure or a set of rules for solving a problem or completing a task. It takes an input, performs a series of operations on it, and produces an output. Algorithms are the foundation of computer programming, data processing, and many other areas of problem-solving.

Characteristics of an Algorithm:

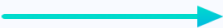
1. **Finiteness:** An algorithm must terminate after a finite number of steps. It should not run indefinitely. If an algorithm does not halt, it's not considered valid.
2. **Definiteness (Unambiguity):** Every step of an algorithm must be precisely defined. The instructions should be clear and unambiguous, so that there is no doubt about what needs to be done at each stage.

3. **Input:** An algorithm takes zero or more inputs, which are the initial data provided to the algorithm for processing. These inputs are essential for the algorithm to produce meaningful results.
4. **Output:** An algorithm must produce at least one output, which is the result of the process. The output must be a solution or a transformation of the input data.
5. **Effectiveness:** The operations in an algorithm must be basic enough to be carried out, in principle, by a human with a pen and paper within a reasonable amount of time. Each step must be computationally feasible.
6. **Generality:** An algorithm should be applicable to a class of problems, not just a specific instance. It must be general enough to handle various inputs and conditions within the problem domain.
7. **Correctness:** An algorithm must produce the correct output for all possible valid inputs. It should be logically sound and guarantee the desired result when executed.

Flowchart:

A flowchart is a diagrammatic representation of a process or workflow, often used to illustrate algorithms and decision-making paths. Flowcharts help visualize complex processes by breaking them down into sequential steps, making them easier to understand and analyze.

Common Symbols Used in Flowchart Design and their Usage:

Symbol	Purpose	Description
	Flow line	Indicates the flow of logic by connecting symbols.
	Terminal (Stop/Start)	Represents the start and the end of a flowchart.
	Input/Output	Used for input and output operation.
	Processing	Used for arithmetic operations and data-manipulations.
	Decision	Used for decision making between two or more alternatives.

	On-page Connector	Used to join different flowline
	Off-page Connector	Used to connect the flowchart portion on a different page.
	Predefined Process/Function	Represents a group of statements performing one processing task.

Examples:

- Develop an algorithm and flowchart to calculate the average of three numbers provided by user.
- Develop an algorithm and flowchart to Display that the any number provided by user is odd or even.
- Develop an algorithm and flowchart to Display that the any number provided by user is prime or composite.
- Develop an algorithm and flowchart to calculate the sum of N natural numbers.
- Develop an algorithm and flowchart to Check if given year is leap year or regular.
- Develop an algorithm and flowchart to calculate the simple interest.